

Actividad 9

Francis Aguilar 22243

Paula Barillas 22764

Bianca Calderón 22272

José Marchena, 22398

Gerardo Pineda 22880

Mónica Salvatierra 22249

Repositorio

<https://github.com/BiancaCalderon/ML-actividad9>

Investigación y adaptación: entrypoints de Python

1. ¿Qué es un entrypoint?

Un entrypoint es metadato de un paquete instalado que anuncia un componente para que otras herramientas lo descubran y utilicen. Tiene un grupo, un nombre y una referencia a un objeto Python. Los grupos `console_scripts` y `gui_scripts` permiten crear comandos; otros grupos permiten descubrir plugins.

En este proyecto:

```
[project.scripts]
transactions-pipeline = "transactions_pipeline.cli:main"
```

- Grupo generado: `console_scripts`.
- Nombre del comando: `transactions-pipeline`.
- Módulo: `transactions_pipeline.cli`.
- Función: `main`.

Al instalar el paquete, el instalador crea un ejecutable que importa esa función y la invoca sin argumentos explícitos. `argparse` lee los argumentos de la terminal. El comportamiento equivale a `sys.exit(main())`: cero representa éxito y una excepción no controlada produce una salida distinta de cero.

`if __name__ == "__main__"` permite ejecutar un módulo directamente, pero por sí solo no registra metadatos ni crea un comando instalado. Tampoco debe confundirse un entrypoint de empaquetado con la instrucción `ENTRYPOINT` de Docker.

2. Adaptación realizada

El ZIP original ya incluía `[project.scripts]` y una CLI con `extract`, `clean` y `train`. Por ello, esta adaptación completa la integración existente:

1. Conserva el entrypoint y las etapas del pipeline de scikit-learn.
2. Añade `run` para ejecutar extracción, limpieza y entrenamiento en orden.
3. Hace explícito el código de salida cero de `main`.
4. Añade `__main__.py` para permitir `python -m transactions_pipeline`.
5. Añade un Makefile que llama al comando instalado.
6. Declara las dependencias directamente en `[project.dependencies]`, conservando las versiones originales, para que gestores como Poetry puedan resolverlas sin depender del mecanismo dinámico de `setuptools`.
7. Ajusta `requires-python` a `>=3.10`, coherente con NumPy 2.2.6 y scikit-learn 1.7.2.

La lógica del modelo permanece en `pipeline.py`. El entrypoint inicia la aplicación; `sklearn.pipeline.Pipeline` encadena los transformadores y el estimador dentro del entrenamiento. Son responsabilidades diferentes y complementarias.

3. Instalación y ejecución

Desde `ML-Taller2-main/taller2/transactions-pipeline`, con Python 3.10 o superior:

```
python -m venv .venv
# Linux/macOS o WSL:
source .venv/bin/activate
# PowerShell en Windows: .venv\Scripts\Activate.ps1
python -m pip install -e .
transactions-pipeline --help
transactions-pipeline run /ruta/transacciones.csv --log-file logs/training.log
```

También se pueden ejecutar las etapas individualmente:

```
transactions-pipeline extract /ruta/transacciones.csv --out data/extracted.csv
transactions-pipeline clean --input data/extracted.csv --out data/cleaned.csv --features-out
transactions-pipeline train --input data/cleaned.csv --features data/features.json --model-out
```

`run` utiliza `data/` para los CSV intermedios y `features.json`, y `models/model.joblib` para el mejor pipeline entrenado. `--work-dir` y `--model-out` permiten cambiar esas rutas. Las rutas relativas se resuelven respecto al directorio desde el cual se ejecuta el comando. El log registra el entrenamiento.

4. ¿Qué usos tendría en producción?

Un entrypoint permite ofrecer una interfaz estable para iniciar tareas sin conocer la ubicación interna de los archivos Python. En este caso, un servidor puede ejecutar `transactions-pipeline run ...` para entrenar un modelo por lotes.

Aplicaciones concretas:

- Programar ejecuciones con cron, un planificador o un orquestador.
- Iniciar procesos dentro de contenedores o trabajos de CI/CD.

- Distribuir la aplicación como wheel e instalar el mismo comando en distintos servidores.
- Integrar los códigos de salida con sistemas que detectan fallos y deciden si reintentar.
- Descubrir extensiones mediante grupos de plugins, si la aplicación implementa ese mecanismo.

Estos usos son aplicaciones del mecanismo descrito por PyPA. El entrypoint facilita la invocación; la reproducibilidad también requiere controlar versiones, datos y configuración. No implementa por sí mismo planificación, reintentos ni monitoreo.

5. ¿Se puede usar con Conda, uv, Poetry y pip?

Sí. El entrypoint pertenece al paquete y sus metadatos, mientras que el gestor instala el paquete o administra el entorno donde se ejecuta.

Herramienta	Instalación desde la carpeta del paquete	Ejecución
pip	<code>python -m pip install -e .</code>	<code>transactions-pipeline --help</code>
uv	<code>uv sync</code>	<code>uv run transactions-pipeline --help</code>
Poetry 2.x	<code>poetry install</code>	<code>poetry run transactions-pipeline --help</code>
Conda + pip	Crear y activar un entorno Conda con Python y pip; luego <code>python -m pip install -e .</code>	<code>transactions-pipeline --help</code>

Ejemplo Conda:

```
conda create -n taller-entrypoints python=3.11 pip
conda activate taller-entrypoints
python -m pip install -e .
transactions-pipeline --help
```

Conda no instala directamente este `pyproject.toml` mediante `conda install`. ∴ aquí se utiliza pip dentro del entorno. Para distribuir un paquete Conda nativo se necesita una receta.

La configuración adaptada usa metadatos estándar y mantiene un backend setup-tools. Los comandos uv y Poetry se fundamentan en sus interfaces documentadas; no se probaron en esta ejecución. Después de cambiar un entrypoint conviene

reinstalar el proyecto para regenerar sus ejecutables. El entorno debe estar activo o el comando debe ejecutarse mediante su gestor para que el ejecutable sea accesible.

6. ¿Qué relación tiene con un Makefile?

Un Makefile define objetivos, dependencias y recetas de comandos. Puede invocar el entrypoint como cualquier programa. El entrypoint expone la aplicación Python y Make organiza tareas del proyecto.

Ejemplo del Makefile incluido:

```
CLI ?= transactions-pipeline
CSV ?= transactions.csv

.PHONY: run
run:
    $(CLI) run "$$(CSV)"
```

La línea de la receta debe comenzar con un tabulador. Para ejecutarla:

```
make install
make run CSV=/ruta/transacciones.csv
```

El archivo completo también incluye `extract`, `clean` y `train`. Sus dependencias hacen que `make train` ejecute primero `extract` y luego `clean`. Como son objetivos `.PHONY`, las etapas se ejecutan nuevamente en cada invocación; este Makefile no implementa caché por fechas de archivos.

Con otros gestores se puede cambiar la variable `CLI`:

```
make run CLI="uv run transactions-pipeline" CSV=/ruta/transacciones.csv
make run CLI="poetry run transactions-pipeline" CSV=/ruta/transacciones.csv
```

Un trabajo de GitHub Actions puede instalar el paquete y llamar `make run`. Así, el mismo objetivo sirve para automatización local y CI. Make requiere estar instalado; en Windows se puede usar WSL.

Referencias

- PyPA. Entry points specification: <https://packaging.python.org/en/latest/specifications/entry-points/>
- uv. Configuring projects, Entry points: <https://docs.astral.sh/uv/concepts/projects/config/>
- Poetry. The pyproject.toml file, scripts: <https://python-poetry.org/docs/pyproject/#scripts>
- Conda. Managing packages, Installing non-conda packages: <https://docs.conda.io/projects/conda/en/latest/guide/tasks/manage-pkgs.html>
- DataCamp. GitHub Actions and MakeFile: A Hands-on Introduction (referencia de la consigna): <https://www.datacamp.com/tutorial/makefile-github-actions-tutorial>

La explicación técnica de entypoints se basa en PyPA, y la integración de uv, Poetry y Conda en la documentación oficial de esas herramientas. La propuesta de automatización es una aplicación al proyecto adjunto.